

02. Density-Based Learning: DBSCAN and HDBSCAN

Algorithms Covered:

1. **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise)
2. **HDBSCAN** (Hierarchical Density-Based Spatial Clustering of Applications with Noise)

Core Focus:

- Discovering clusters of arbitrary shapes and variable densities
- Automatic anomaly and noise identification
- Systematic hyperparameter selection (k -distance graph for ϵ and `min_samples`)
- Comparative benchmark against centroid-based clustering (K-Means)

```
In [1]: # ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Density-based clustering algorithms + centroid baseline ----
from sklearn.cluster import DBSCAN, HDBSCAN, KMeans
# ---- Synthetic dataset generators for non-convex shapes ----
from sklearn.datasets import make_moons, make_blobs, make_circles
# ---- Nearest-neighbor search (used to pick epsilon) and scaling ----
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler
# ---- Evaluation metrics ----
from sklearn.metrics import silhouette_score, adjusted_rand_score

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available
else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("Density-based libraries loaded successfully!")
```

Density-based libraries loaded successfully!

1. Generating Complex Non-Spherical Benchmark Data

Real-world spatial, geographic, and sensor data often exhibits arbitrary shapes, varying local densities, and background noise where centroid-based algorithms (like K-Means) fail completely. Here we construct a composite benchmark dataset featuring:

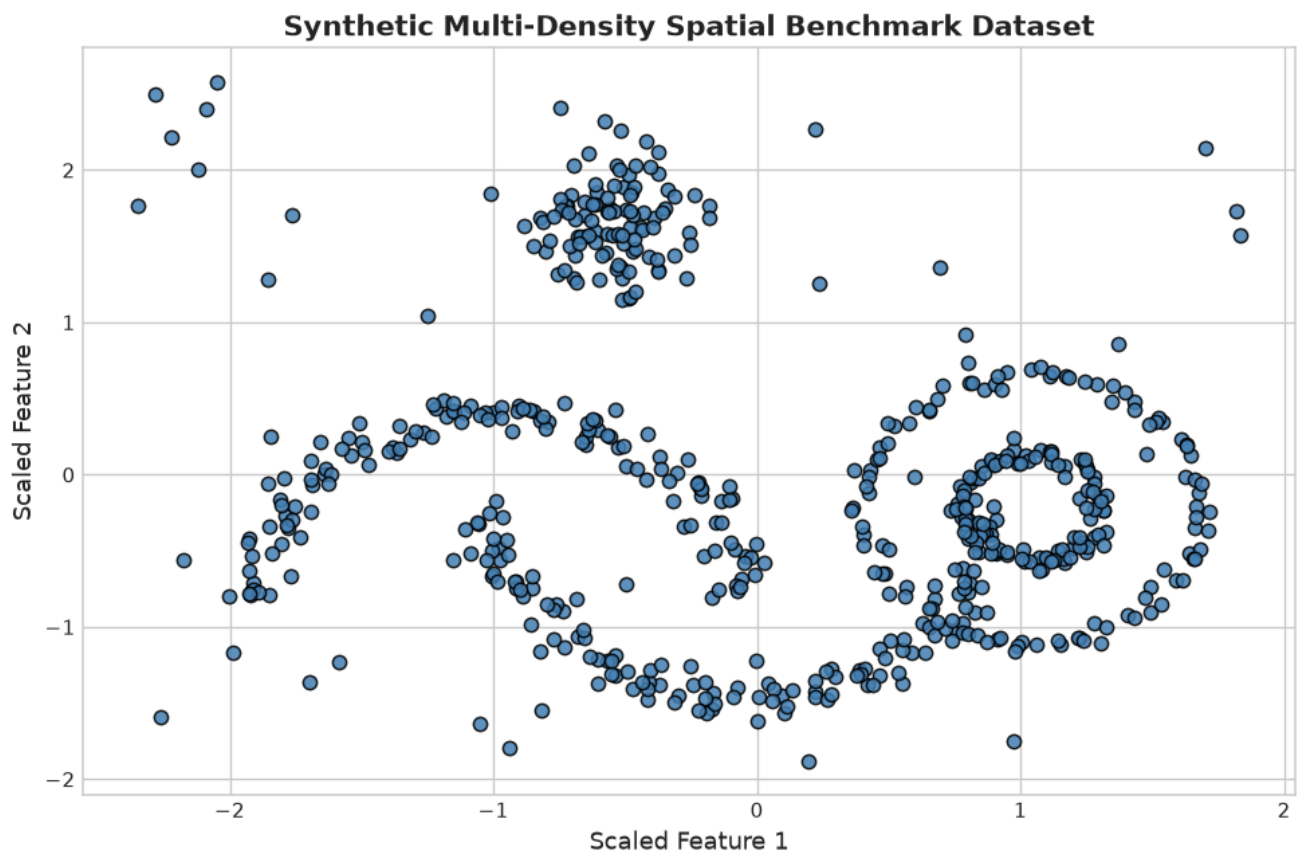
- Two interleaving half moons
- A dense circular core
- A sparser Gaussian blob
- Random uniform background noise (outliers)

```
In [2]: # ---- Build a composite benchmark dataset with complex geometry ----
# Two interleaving half-moons (non-convex)
n_samples = 600
X_moons, y_moons = make_moons(n_samples=250, noise=0.06, random_state=42)
# Two concentric circles, shifted to the right so they do not overlap the moons
X_circle, y_circle = make_circles(n_samples=200, noise=0.04, factor=0.4, random_state=42)
X_circle = X_circle * 0.7 + np.array([2.2, 0.5])
# One sparse Gaussian blob above the moons
X_blob, y_blob = make_blobs(n_samples=100, centers=[[0.5, 2.0]], cluster_std=0.2,
random_state=42)
# Uniform background noise (outliers) spanning the whole area
X_noise = np.random.uniform(low=[-1.5, -1.0], high=[3.2, 2.8], size=(50, 2))

# Stack all point groups into one feature matrix
X_raw = np.vstack([X_moons, X_circle, X_blob, X_noise])
# Ground-truth labels for benchmarking (noise points are marked with -1)
y_true = np.concatenate([y_moons, y_circle + 2, y_blob + 4, -np.ones(50, dtype=int)])

# Standardize so that epsilon (a distance threshold) has a comparable meaning in both axes
scaler = StandardScaler()
X = scaler.fit_transform(X_raw)

# ---- Plot the raw benchmark to show the shapes/noise the algorithms must handle ----
plt.figure(figsize=(9, 6))
plt.scatter(X[:, 0], X[:, 1], c='steelblue', edgecolors='k', s=45, alpha=0.85)
plt.title("Synthetic Multi-Density Spatial Benchmark Dataset", fontsize=14,
fontweight='bold')
plt.xlabel("Scaled Feature 1", fontsize=12)
plt.ylabel("Scaled Feature 2", fontsize=12)
plt.tight_layout()
plt.show()
```



2. Why Centroid-Based Clustering (K-Means) Fails on

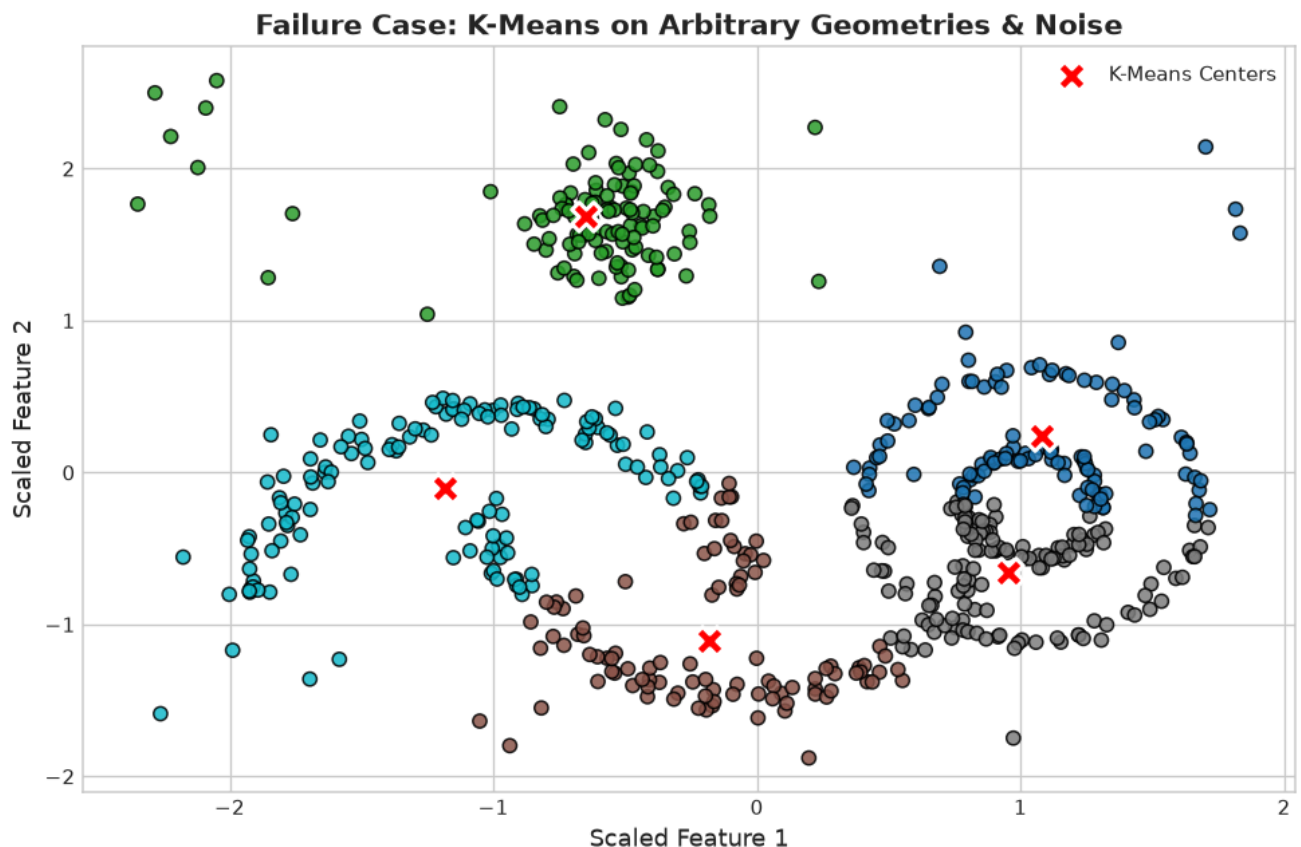
Non-Convex Geometries

K-Means assumes clusters are spherical and isotropic, minimizing Euclidean variance around centroids. Let's see what happens when K-Means is applied to this non-convex dataset.

```
In [3]: # ---- Baseline: apply K-Means (centroid-based) to the non-convex data ----
# K-Means assumes spherical, equally sized clusters, so it cannot follow curved shapes.
km = KMeans(n_clusters=5, random_state=42, n_init=10)
km_labels = km.fit_predict(X)

# Visualize where K-Means places its straight-Line Voronoi boundaries
plt.figure(figsize=(9, 6))
plt.scatter(X[:, 0], X[:, 1], c=km_labels, cmap='tab10', edgecolors='k', s=45, alpha=0.85)
plt.scatter(km.cluster_centers_[:, 0], km.cluster_centers_[:, 1], c='red', marker='X',
s=200, edgecolors='white', linewidths=2, label='K-Means Centers')
plt.title("Failure Case: K-Means on Arbitrary Geometries & Noise", fontsize=14,
fontweight='bold')
plt.xlabel("Scaled Feature 1", fontsize=12)
plt.ylabel("Scaled Feature 2", fontsize=12)
plt.legend()
plt.tight_layout()
plt.show()

# Adjusted Rand Index measures agreement with the ground truth (1.0 = perfect, 0 = chance)
print(f"K-Means Adjusted Rand Index (ARI vs ground truth): {adjusted_rand_score(y_true,
km_labels):.4f}")
```



K-Means Adjusted Rand Index (ARI vs ground truth): 0.4426

3. Algorithm 1: DBSCAN

Mathematical Concepts

DBSCAN defines clusters as continuous regions of high density separated by regions of low density. For a dataset D :

1. **ϵ -Neighborhood:**

$$N_{\epsilon}(p) = \{q \in D \mid \text{dist}(p, q) \leq \epsilon\}$$

2. **Core Point:** A point p is a core point if $|N_{\epsilon}(p)| \geq \text{MinPts}$.
3. **Directly Density-Reachable:** Point q is directly density-reachable from p if $q \in N_{\epsilon}(p)$ and p is a core point.
4. **Density-Reachable & Density-Connected:** Points connected through chains of core points form a cluster.
5. **Noise / Outliers:** Any point not reachable from any core point is assigned label -1 .

Hyperparameter Selection via the k -Distance Graph

A standard heuristic for choosing ϵ is plotting the distance to the k -th nearest neighbor sorted in ascending order (where $k = \text{MinPts}$). The "knee" or "elbow" of maximum curvature indicates the threshold where point density sharply drops into noise.

Step-by-Step Algorithm (DBSCAN)

1. **Parameters:** radius ϵ and minimum points MinPts (here 0.22 and 6).
2. For every point count its ϵ -neighborhood; points with $\geq \text{MinPts}$ neighbors become **core points**.
3. **Grow clusters:** connect core points that lie within ϵ of each other (density-reachability chain).
4. Attach border points (within ϵ of a core point) to that core's cluster.
5. Label every point that is neither core nor border as **noise (-1)**.
6. **Output:** arbitrary-shaped clusters + explicit noise labels. No K is required.

How ϵ and MinPts Are Chosen Here

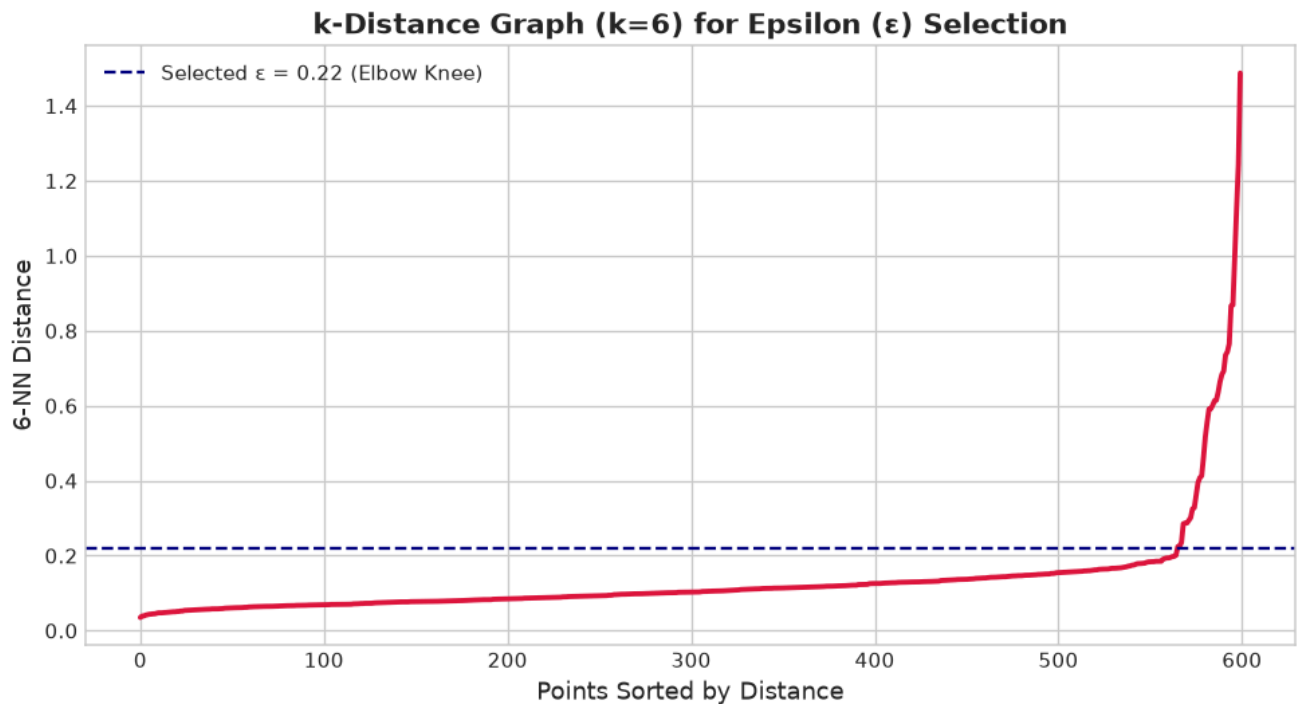
1. Set MinPts $\approx 2 \times \text{dimensions}$.
2. Plot the sorted distance to the MinPts-th neighbor and pick the "knee" as ϵ .
3. Fit DBSCAN and inspect cluster count, noise %, and agreement with the ground truth.

```
In [4]: # ---- Choosing epsilon from the k-distance graph ----
# Heuristic: sort the distance of each point to its k-th nearest neighbor.
# The "knee" of the curve marks the transition from dense clusters to sparse noise,
# which is a good value for epsilon.
min_samples = 6 # MinPts rule of thumb: at least 2 * number of dimensions
nbrs = NearestNeighbors(n_neighbors=min_samples).fit(X)
distances, indices = nbrs.kneighbors(X) # distance to each point's k neighbors

k_distances = np.sort(distances[:, -1]) # k-th neighbor distance, sorted ascending

plt.figure(figsize=(9, 5))
plt.plot(k_distances, color='crimson', linewidth=2.5)
plt.title(f"k-Distance Graph (k={min_samples}) for Epsilon (ε) Selection", fontsize=14,
fontweight='bold')
```

```
plt.xlabel("Points Sorted by Distance", fontsize=12)
plt.ylabel(f"{min_samples}-NN Distance", fontsize=12)
plt.axhline(y=0.22, color='navy', linestyle='--', label='Selected  $\epsilon = 0.22$  (Elbow Knee)')
plt.legend()
plt.tight_layout()
plt.show()
```



```
In [5]: # ---- Fit DBSCAN with the chosen epsilon and MinPts ----
# DBSCAN grows clusters from core points (>= min_samples neighbors within eps),
# connects cores via density-reachability, and labels everything else as noise (-1).
eps_selected = 0.22
dbscan = DBSCAN(eps=eps_selected, min_samples=min_samples)
db_labels = dbscan.fit_predict(X)

# Identify which samples are core points (returned by the model) vs. border points
core_samples_mask = np.zeros_like(db_labels, dtype=bool)
core_samples_mask[dbscan.core_sample_indices_] = True

# Count clusters (labels != -1) and noise points
n_clusters_db = len(set(db_labels)) - (1 if -1 in db_labels else 0)
n_noise_db = np.sum(db_labels == -1)

print(f"DBSCAN Results:")
print(f" - Estimated number of clusters: {n_clusters_db}")
print(f" - Number of noise points: {n_noise_db} ({n_noise_db / len(X) * 100:.1f}%)")
print(f" - Adjusted Rand Index (ARI): {adjusted_rand_score(y_true, db_labels):.4f}")
```

DBSCAN Results:

- Estimated number of clusters: 3
- Number of noise points: 28 (4.7%)
- Adjusted Rand Index (ARI): 0.4637

```
In [6]: # ---- Visualize DBSCAN: core points, border points and noise ----
plt.figure(figsize=(10, 6))

unique_labels = set(db_labels)
colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1, len(unique_labels))]

for k, col in zip(unique_labels, colors):
    if k == -1:
        # Black is used for noise points
```

```

col = [0, 0, 0, 1]

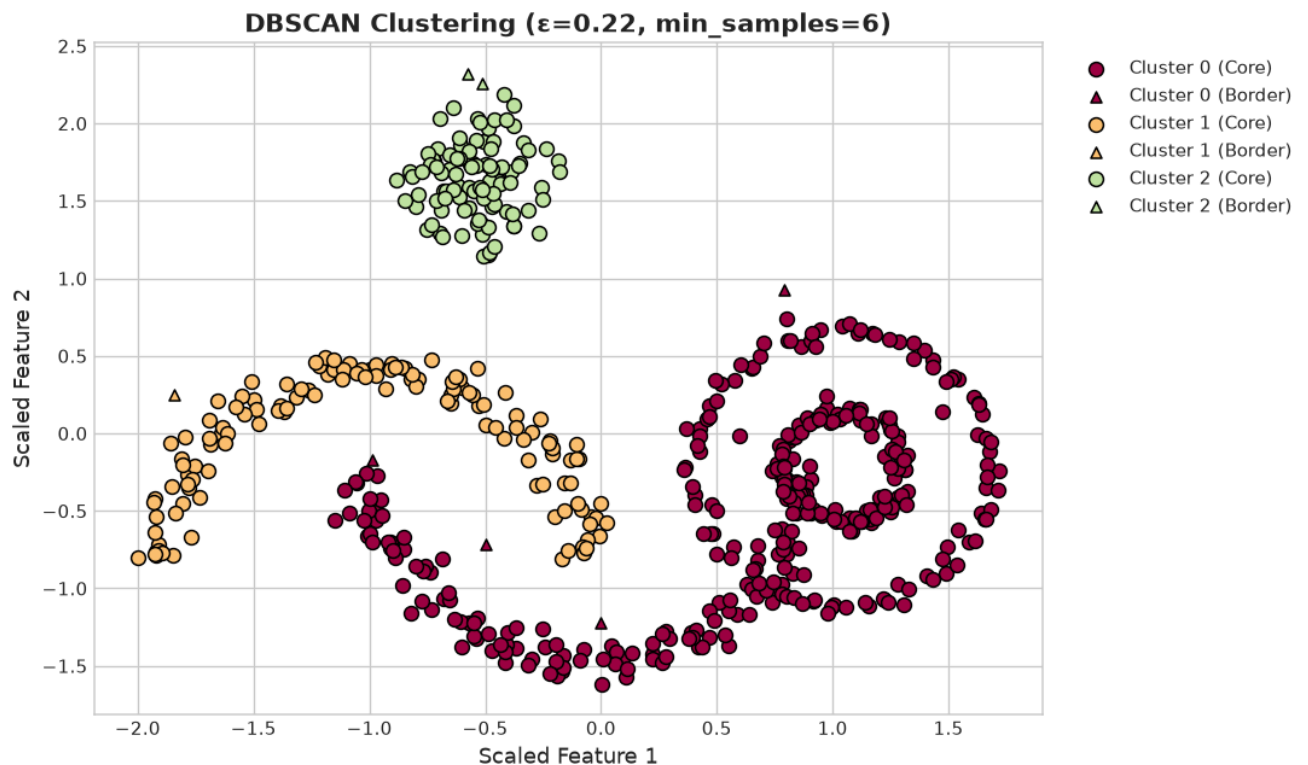
class_member_mask = (db_labels == k)

# Core points of this cluster (large circles)
xy_core = X[class_member_mask & core_samples_mask]
if len(xy_core) > 0:
    plt.scatter(xy_core[:, 0], xy_core[:, 1], color=tuple(col), edgecolors='k', s=60,
label=f'Cluster {k} (Core)' if k != -1 else 'Noise')

# Border points of this cluster (small triangles) - not core, but inside eps of a core
xy_border = X[class_member_mask & ~core_samples_mask]
if len(xy_border) > 0 and k != -1:
    plt.scatter(xy_border[:, 0], xy_border[:, 1], color=tuple(col), edgecolors='k',
marker='^', s=35, label=f'Cluster {k} (Border)')

plt.title(f"DBSCAN Clustering (ε={eps_selected}, min_samples={min_samples})", fontsize=14,
fontweight='bold')
plt.xlabel("Scaled Feature 1", fontsize=12)
plt.ylabel("Scaled Feature 2", fontsize=12)
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()

```



4. Algorithm 2: HDBSCAN

Mathematical Concepts

While DBSCAN uses a single fixed radius ϵ across the entire space (struggling when clusters possess varying densities), **HDBSCAN** converts DBSCAN into a hierarchical clustering algorithm and extracts the most persistent flat clusters:

1. **Core Distance:** $d_{\text{core}}(p)$ is the distance from p to its k -th nearest neighbor.
2. **Mutual Reachability Distance:**

$$d_{\text{mreach}}(a, b) = \max\{d_{\text{core}}(a), d_{\text{core}}(b), d(a, b)\}$$

3. **Minimum Spanning Tree & Condensed Tree:** A tree is built using mutual reachability as weights, then condensed by tracking cluster births ($\lambda_{\text{birth}} = 1/\epsilon$) and deaths (λ_{death}).

4. **Cluster Stability:**

$$S(C_i) = \sum_{p \in C_i} (\lambda_p - \lambda_{\text{birth}}(C_i))$$

Clusters with highest accumulated persistence stability are retained.

Step-by-Step Algorithm (HDBSCAN)

1. Compute the **core distance** $d_{\text{core}}(p)$ = distance from p to its k -th nearest neighbor.
2. Define the **mutual reachability distance**: $d_{\text{mreach}}(a,b) = \max\{d_{\text{core}}(a), d_{\text{core}}(b), d(a,b)\}$.
3. Build a **minimum spanning tree** on the mutual-reachability graph.
4. Convert the MST into a **cluster hierarchy** and then into a **condensed tree** (cluster births/deaths).
5. Score each candidate cluster by **stability** $S(C) = \sum (\lambda_p - \lambda_{\text{birth}}(C))$ and keep the most stable ones.
6. **Output:** clusters of varying density, membership probabilities, and noise labels - with no global ϵ .

```
In [7]: # ---- Fit HDBSCAN (no global epsilon required) ----
# HDBSCAN builds a hierarchy over all density levels and keeps the most stable clusters;
# min_cluster_size sets the smallest allowed cluster, min_samples controls conservativeness.
hdb = HDBSCAN(min_cluster_size=15, min_samples=6, store_centers='centroid', copy=False)
hdb_labels = hdb.fit_predict(X)

# Count clusters and noise points exactly as for DBSCAN
n_clusters_hdb = len(set(hdb_labels)) - (1 if -1 in hdb_labels else 0)
n_noise_hdb = np.sum(hdb_labels == -1)

print(f"HDBSCAN Results:")
print(f" - Estimated number of clusters: {n_clusters_hdb}")
print(f" - Number of noise points: {n_noise_hdb} ({n_noise_hdb / len(X) * 100:.1f}%)")
print(f" - Adjusted Rand Index (ARI): {adjusted_rand_score(y_true, hdb_labels):.4f}")
```

HDBSCAN Results:

- Estimated number of clusters: 3
- Number of noise points: 22 (3.7%)
- Adjusted Rand Index (ARI): 0.4578

```
In [8]: # ---- Visualize HDBSCAN results and soft membership probabilities ----
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))

# Panel 1: hard cluster assignments
palette = sns.color_palette("tab10", n_clusters_hdb)
for cluster_id in range(n_clusters_hdb):
    pts = X[hdb_labels == cluster_id]
    ax1.scatter(pts[:, 0], pts[:, 1], color=palette[cluster_id], edgecolors='k', s=50,
label=f'Cluster {cluster_id}', alpha=0.85)

# Noise points marked with black crosses
noise_pts = X[hdb_labels == -1]
ax1.scatter(noise_pts[:, 0], noise_pts[:, 1], color='black', marker='x', s=40, label='Noise
(-1)', alpha=0.7)
ax1.set_title(f"HDBSCAN Cluster Assignments (Clusters: {n_clusters_hdb})", fontsize=13,
fontweight='bold')
ax1.set_xlabel("Scaled Feature 1")
```



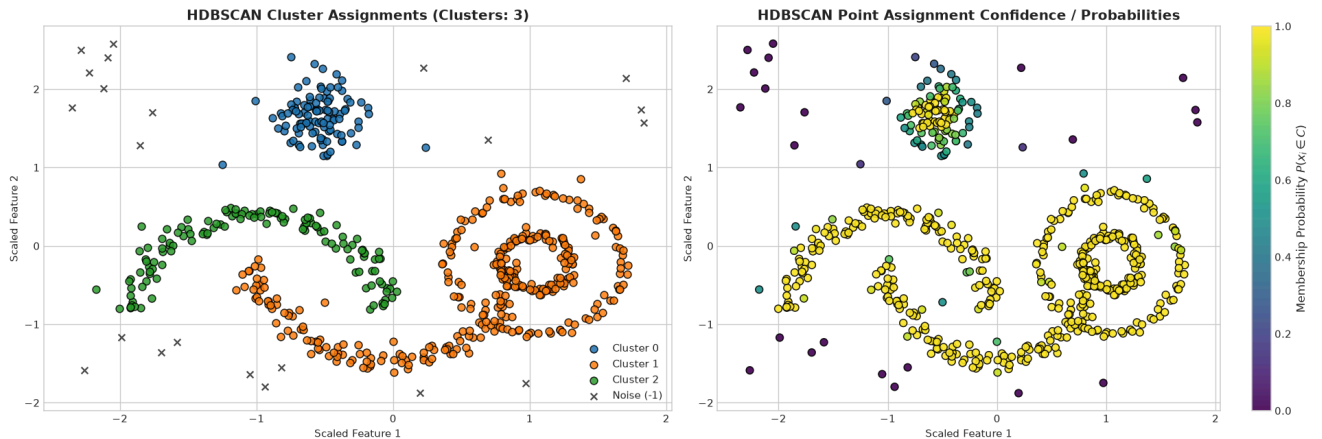
```

ax1.set_ylabel("Scaled Feature 2")
ax1.legend()

# Panel 2: membership probability per point (how confidently it belongs to its cluster)
sc = ax2.scatter(X[:, 0], X[:, 1], c=hdb.proBABILITIES_, cmap='viridis', edgecolors='k',
s=50, alpha=0.9)
cbar = plt.colorbar(sc, ax=ax2)
cbar.set_label(r"Membership Probability  $P(x_i \in C)$ ", fontsize=11)
ax2.set_title("HDBSCAN Point Assignment Confidence / Probabilities", fontsize=13,
fontweight='bold')
ax2.set_xlabel("Scaled Feature 1")
ax2.set_ylabel("Scaled Feature 2")

plt.tight_layout()
plt.show()

```



5. Comparative Evaluation: K-Means vs DBSCAN vs HDBSCAN

Let's synthesize our empirical findings across all three algorithms:

- **Adjusted Rand Index (ARI)** measures ground-truth alignment (adjusted for chance, max 1.0).
- **Noise Detection** capability.
- **Silhouette Score** evaluated strictly on clustered non-noise points.

Evaluation Process

1. Collect labels from K-Means, DBSCAN and HDBSCAN on the same benchmark data.
2. Compute ARI vs. ground truth (chance-adjusted agreement), cluster count, noise %, and silhouette on non-noise points.
3. Compare all three in one table to show when centroid-based clustering fails and density-based clustering succeeds.

```

In [9]: # ---- Compare K-Means, DBSCAN and HDBSCAN on equal footing ----
def eval_density_algo(name, labels):
    """Compute summary metrics for one density-based partition."""
    mask = (labels != -1) # ignore noise for silhouette
    sil = silhouette_score(X[mask], labels[mask]) if len(set(labels[mask])) > 1 else np.nan
    ari = adjusted_rand_score(y_true, labels) # agreement with ground truth
    n_cl = len(set(labels)) - (1 if -1 in labels else 0) # cluster count
    noise_pct = (np.sum(labels == -1) / len(labels)) * 100 # % labelled as noise
    return {
        "Algorithm": name,
        "Clusters Found": n_cl,

```



```

    "Noise Points (%)": f"{noise_pct:.1f}%",
    "ARI vs Ground Truth": ari,
    "Silhouette (Non-Noise)": sil
}

# Build one table row per algorithm
comparison_table = pd.DataFrame([
    eval_density_algo("K-Means", km_labels),
    eval_density_algo("DBSCAN", db_labels),
    eval_density_algo("HDBSCAN", hdb_labels)
]).set_index("Algorithm")

# Highlight the best ARI/silhouette values in green
display(comparison_table.style.highlight_max(subset=['ARI vs Ground Truth', 'Silhouette (Non-Noise)'], color='lightgreen'))

```

	Clusters Found	Noise Points (%)	ARI vs Ground Truth	Silhouette (Non-Noise)
Algorithm				
K-Means	5	0.0%	0.442594	0.448428
DBSCAN	3	4.7%	0.463683	0.434245
HDBSCAN	3	3.7%	0.457798	0.433431

6. Summary & Algorithmic Takeaways

1. **DBSCAN** excels at extracting arbitrary non-linear shapes and separating anomalies, provided the density throughout the clusters is relatively uniform.
2. **HDBSCAN** eliminates the fragile tuning of a global ϵ parameter and naturally handles multi-density clusters while producing calibrated cluster membership probabilities.
3. **K-Means** is computationally fast but fundamentally inappropriate for complex topological structures, non-spherical clusters, or noisy datasets.